

AWS VPC PEERING CONFIGURATION

AWS Networking Project Documentation

1. Introduction

Amazon Virtual Private Cloud (VPC) enables users to create isolated virtual networks within the AWS Cloud. VPC Peering is a networking connection that allows two VPCs to communicate with each other using private IP addresses. In this project, two independent AWS VPCs are configured and connected using a VPC Peering Connection. Route tables and security group rules are configured to enable communication between resources deployed in both VPCs.

2. Project Objective

The primary objective of this project is to establish private network communication between two separate Amazon VPCs using VPC Peering.

- Creation and configuration of two VPCs.
- Creation of subnets within the VPCs.
- Configuration of route tables.
- Creation and acceptance of a VPC Peering Connection.
- Configuration of security group rules.
- Deployment of EC2 instances.
- Testing connectivity between EC2 instances using private IP addresses.

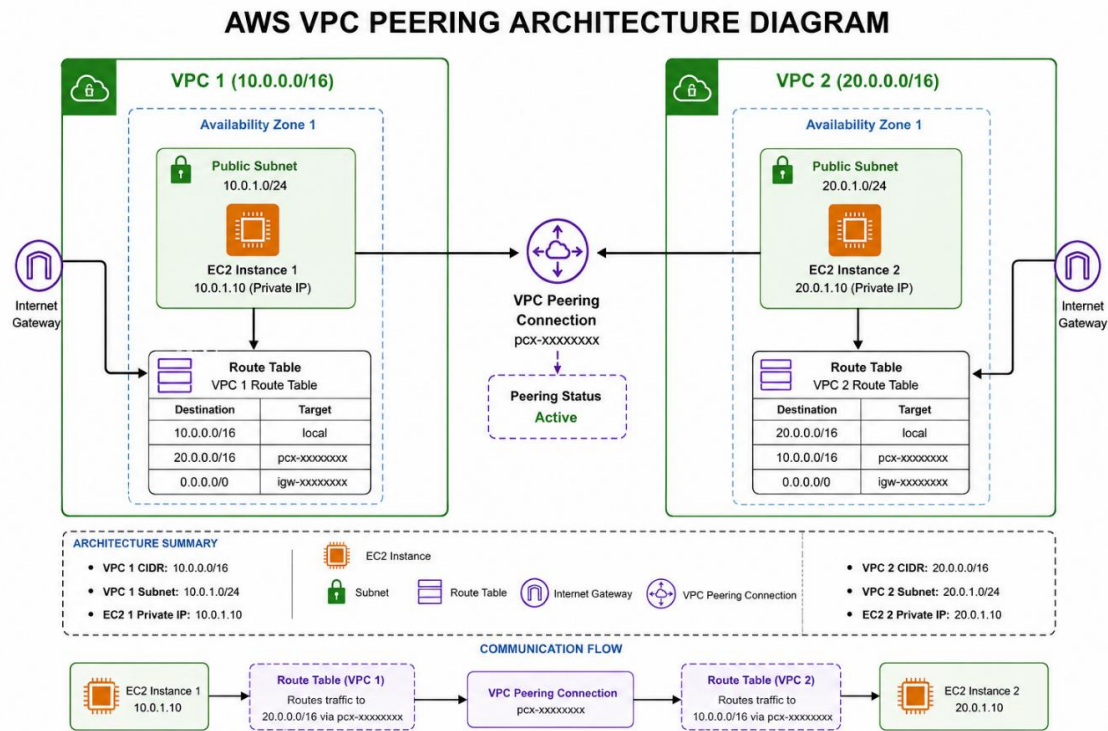
3. AWS Services Used

- Amazon VPC
- VPC Peering
- Amazon EC2
- Subnets
- Route Tables
- Security Groups
- Internet Gateway
- AWS Management Console

4. Network Architecture

The project consists of two separate VPCs connected through a VPC Peering Connection. Each VPC contains its own subnet, route table, security group, and EC2 instance. The route tables are configured to route traffic between the two AWS VPC networks through the VPC Peering Connection.

VPC 1	VPC Peering Connection	VPC 2
Subnet • Route Table • Security Group • EC2	Private Network Path	Subnet • Route Table • Security Group • EC2
Private IP	↔	Private IP



5. VPC Configuration

5.1 VPC 1 Creation

The first VPC was created with a dedicated IPv4 CIDR block. The VPC provides an isolated virtual network environment for the first set of AWS resources.

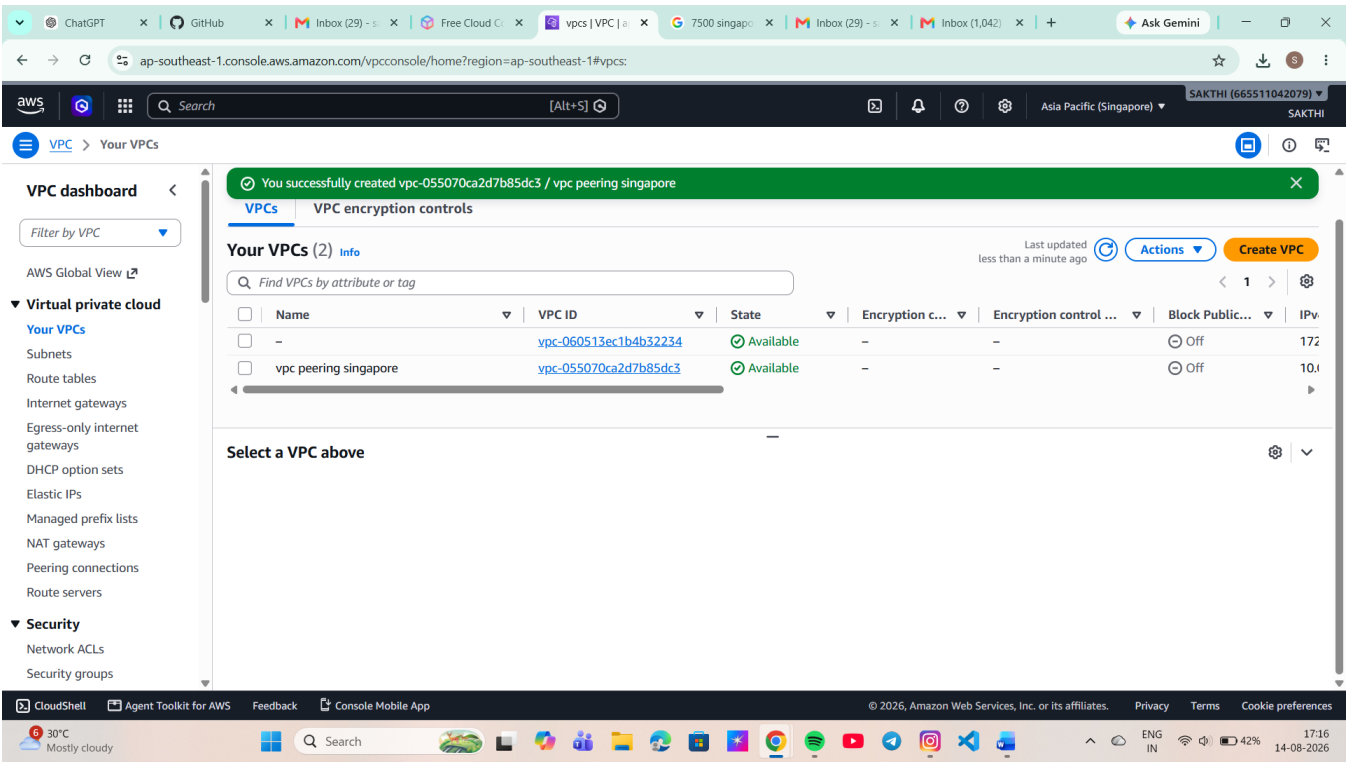


Figure 1: VPC 1 configuration evidence

5.2 VPC 2 Creation

The second VPC was created with a separate and non-overlapping IPv4 CIDR block. Using non-overlapping CIDR ranges allows proper routing between the two VPCs.

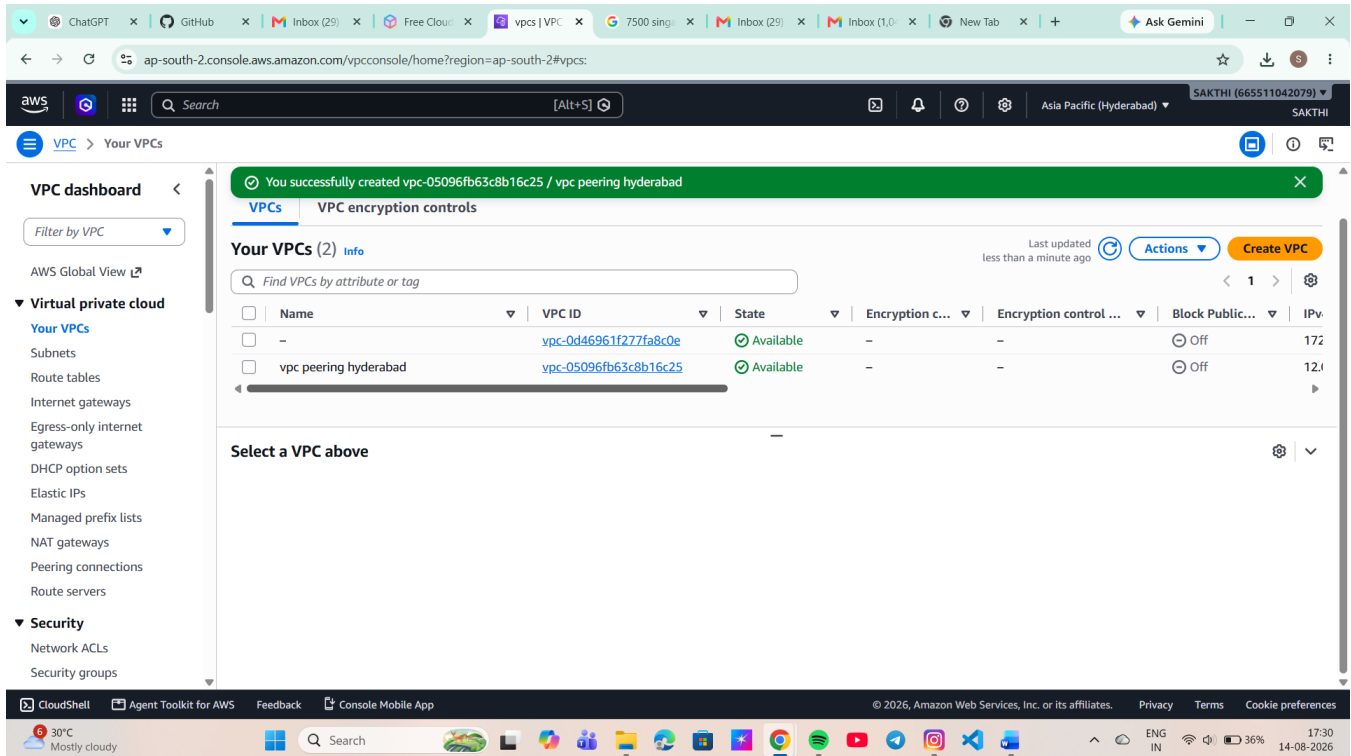


Figure 2: VPC 2 configuration evidence

6. Subnet Configuration

Subnets were created inside the respective VPCs to provide network segments for deploying EC2 instances.

6.1 VPC 1 Subnet

A subnet was created within VPC 1 and associated with the required availability zone.

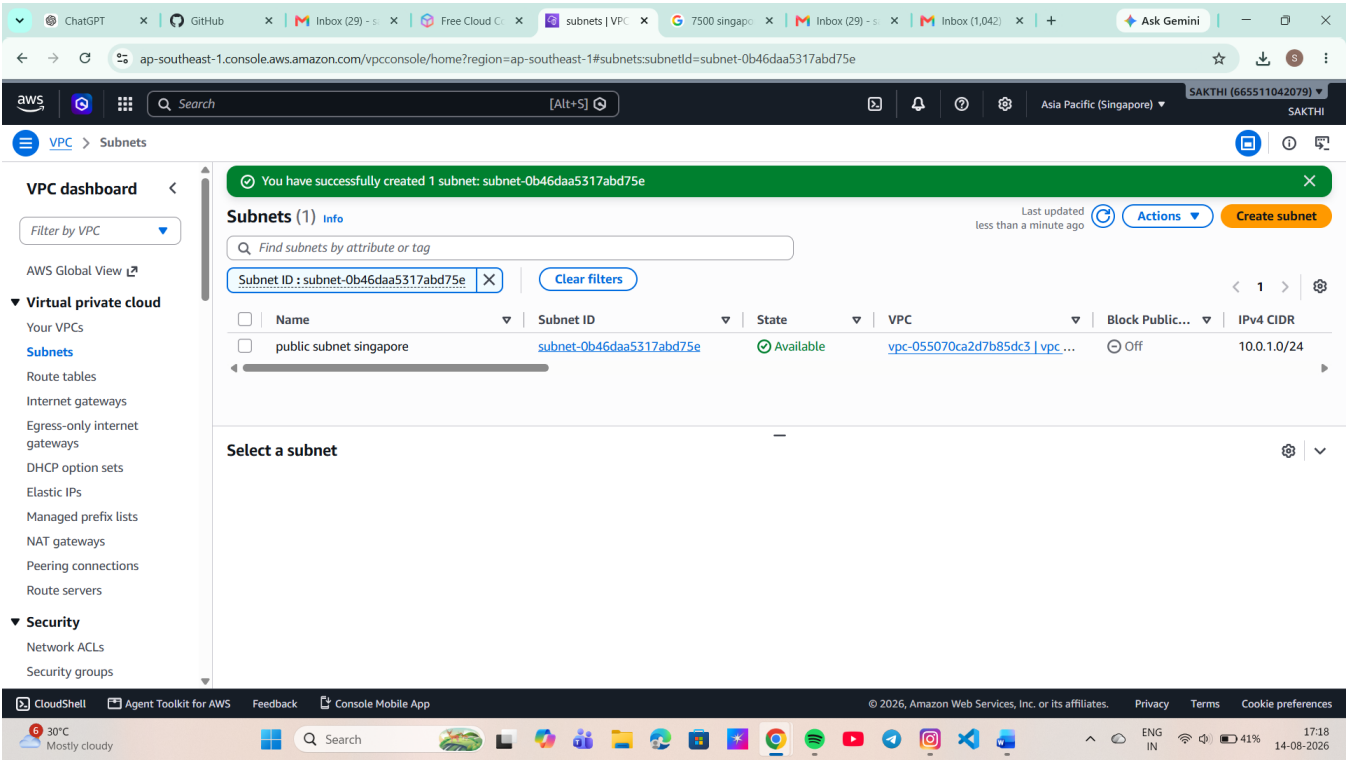


Figure 3: VPC 1 subnet evidence

6.2 VPC 2 Subnet

A subnet was created within VPC 2 for deploying the second EC2 instance.

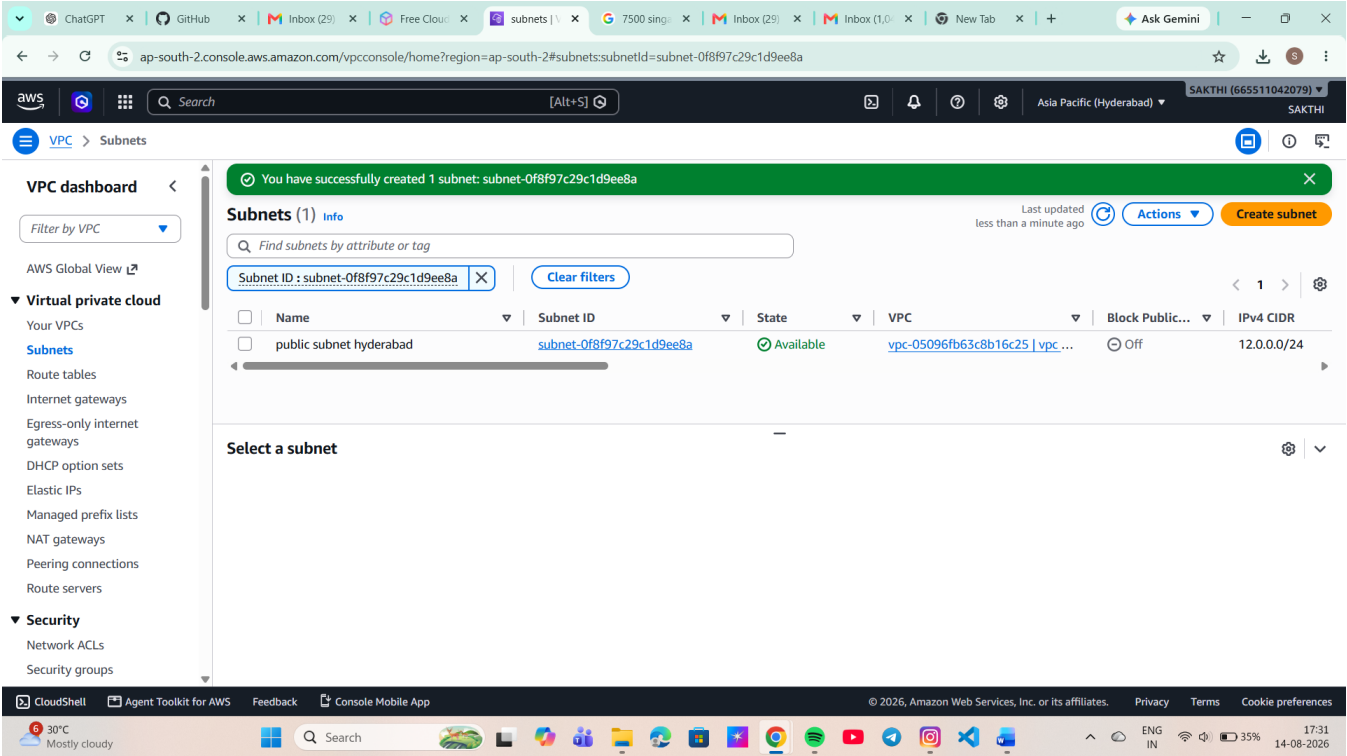


Figure 4: VPC 2 subnet evidence

7. EC2 Instance Configuration

EC2 instances were launched in the respective VPCs to verify communication between the two networks.

7.1 EC2 Instance – VPC 1

An EC2 instance was launched inside the subnet associated with VPC 1.

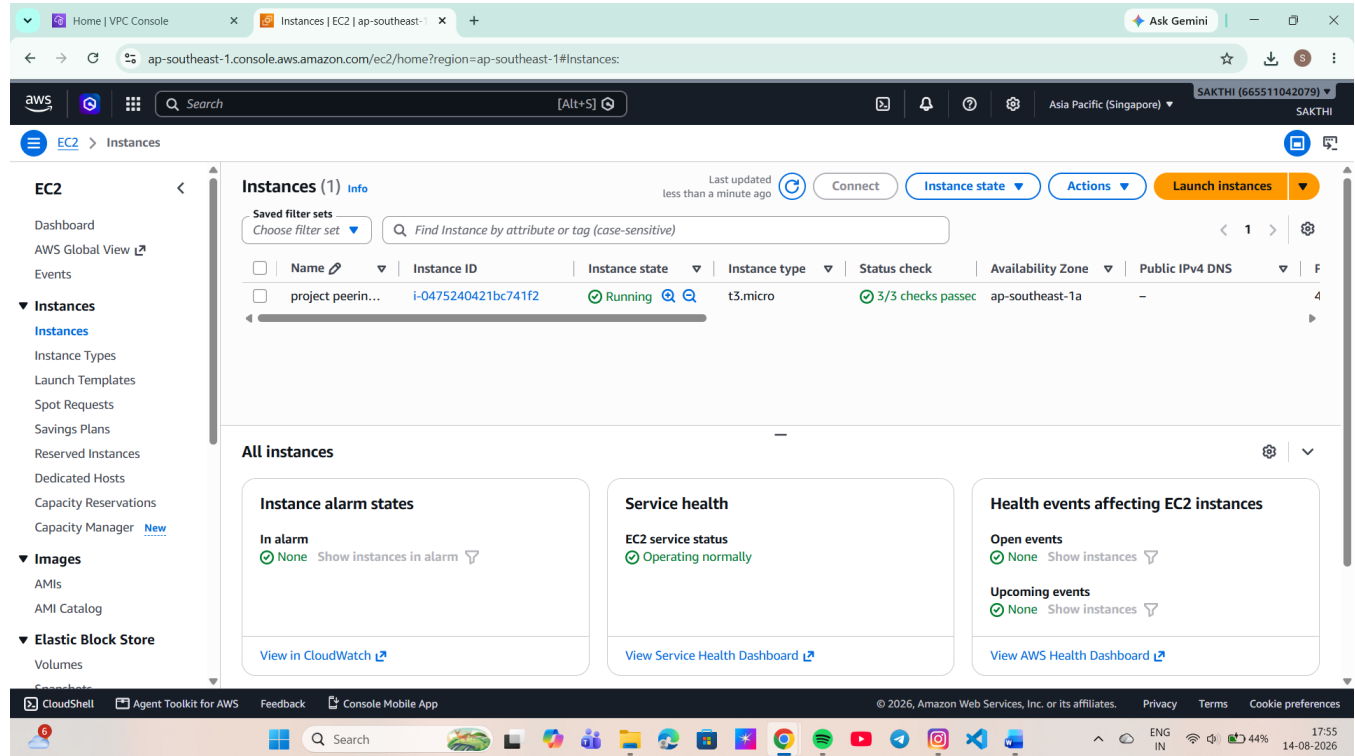


Figure 5: EC2 instance in VPC 1

7.2 EC2 Instance – VPC 2

Another EC2 instance was launched inside the subnet associated with VPC 2.

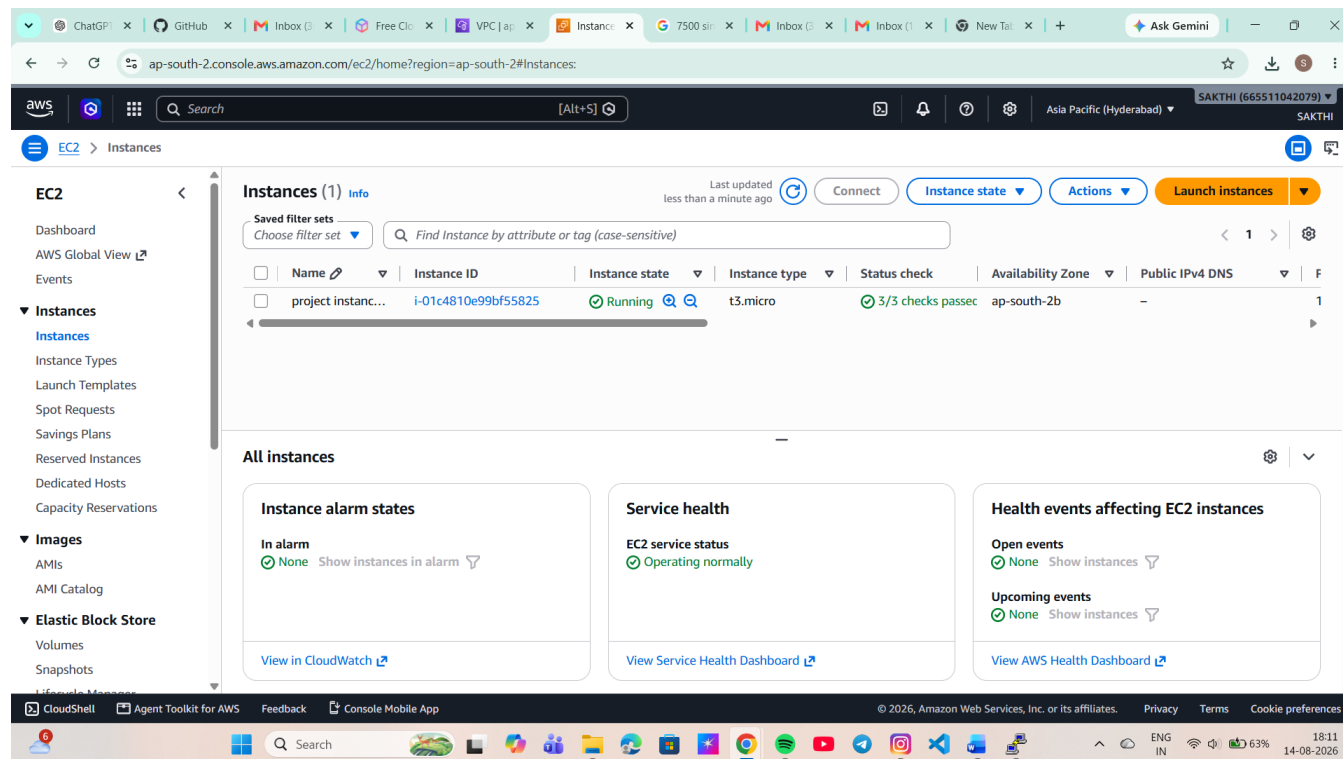


Figure 6: EC2 instance in VPC 2

8. VPC Peering Connection

A VPC Peering Connection was created between VPC 1 and VPC 2. The peering connection provides a private networking path between the two VPCs.

Figure 7: VPC Peering Connection

The peering request was accepted by the peer VPC. After successful acceptance, the peering connection status was verified as Active.

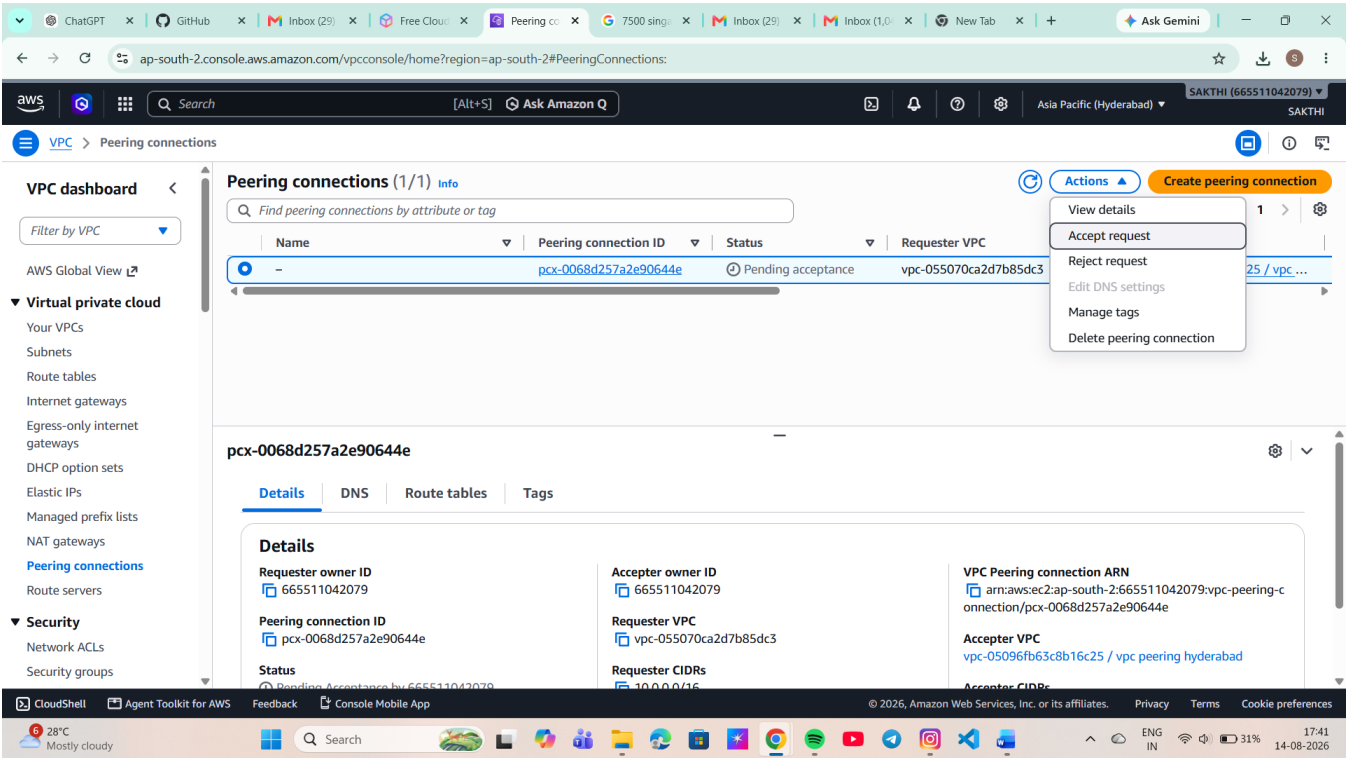


Figure 8: Peering connection acceptance/status

9. Route Table Configuration

Route tables were updated in both VPCs to enable traffic to reach the CIDR block of the peer VPC through the VPC Peering Connection.

9.1 Route Configuration – VPC 1

A route was added to the route table of VPC 1 with the destination set to the CIDR block of VPC 2 and the target configured as the VPC Peering Connection.

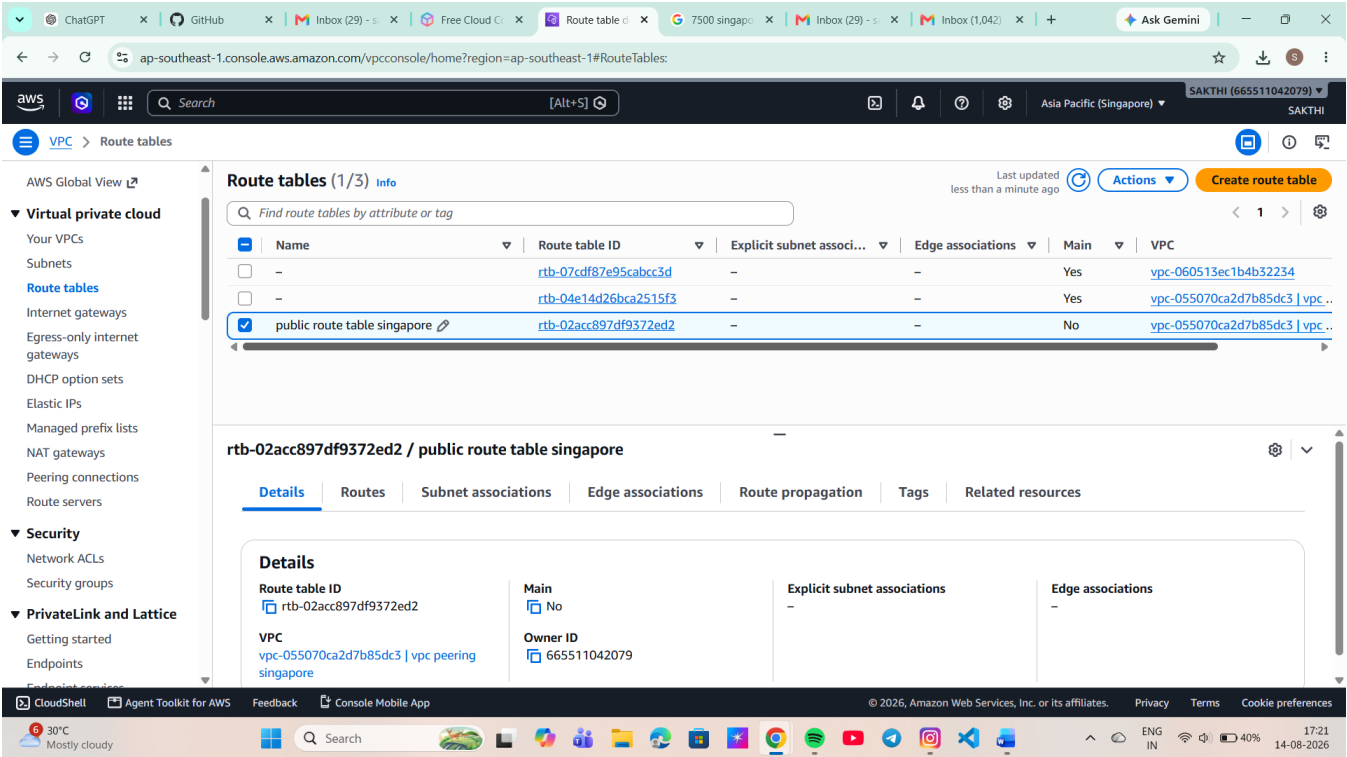


Figure 9: VPC 1 route table

9.2 Route Configuration – VPC 2

Similarly, a route was added to the route table of VPC 2 with the destination set to the CIDR block of VPC 1 and the VPC Peering Connection configured as the target.

Route table **rtb-0f827337a0e7f6f61** | public route table hyderabad was created successfully.

Route tables (4) [Info](#)

Last updated less than a minute ago [Actions](#) [Create route table](#)

Find route tables by attribute or tag

☐	Name	Route table ID	Explicit subnet associ...	Edge associations	Main	VPC
☐	-	rtb-0748dcf6c602738cb	-	-	Yes	vpc-0e9981f73c7babc00
☐	-	rtb-05efe5c1698978085	-	-	Yes	vpc-0d46961f277fa8c0e
☐	public route table hyderabad	rtb-0f827337a0e7f6f61	-	-	No	vpc-05096fb63c8b16c25 vpc
☐	-	rtb-078fd13c91625522b	-	-	Yes	vpc-05096fb63c8b16c25 vpc

Select a route table

Figure 10: VPC 2 route table

10. Security Group Configuration

Security group rules were configured to allow the required network traffic between the EC2 instances. The necessary inbound and outbound rules were configured based on the connectivity requirements of the project.

The screenshot displays the AWS Management Console interface for editing inbound rules on a security group. The page title is "Edit inbound rules" with a sub-header "Inbound rules control the incoming traffic that's allowed to reach the instance." The main content area shows a table of inbound rules. Two rules are listed: one for SSH (port 22) and one for HTTP (port 80), both allowing traffic from 0.0.0.0/0. A warning message at the bottom states: "Rules with source of 0.0.0.0/0 or ::/0 allow all IP addresses to access your instance. We recommend setting security group rules to allow access from known IP addresses only." The console interface includes a top navigation bar with the AWS logo, a search bar, and a breadcrumb trail: EC2 > Security Groups > sg-0929a87446c9f1b52 > Edit inbound rules. The bottom of the screenshot shows a Windows taskbar with various application icons and system tray information.

Security group rule ID	Type	Protocol	Port range	Source	Description - optional
-	SSH	TCP	22	Anywh...	
-	HTTP	TCP	80	Anywh...	

Rules with source of 0.0.0.0/0 or ::/0 allow all IP addresses to access your instance. We recommend setting security group rules to allow access from known IP addresses only.

Buttons: Add rule, Delete, Cancel, Preview changes, Save rules

Figure 11: Security group configuration

11. Connectivity Testing

After completing the VPC Peering and routing configuration, connectivity between the EC2 instances was tested using their private IP addresses. The connectivity test was performed from the EC2 instance in VPC 1 to the EC2 instance in VPC 2. The successful response confirms that network traffic is being routed between the two VPCs through the VPC Peering Connection.

The screenshot displays the AWS Management Console interface. At the top, the browser address bar shows the URL: `ap-southeast-1.console.aws.amazon.com/ec2/home?region=ap-southeast-1#InstanceDetails:instanceId=i-0d06f5e2170bd9356`. The main content area is divided into two panes. The left pane shows the details of an EC2 instance with ID `i-0d06f5e2170bd9356`, including its name `ec2-user@ip-10-0-1-223`, private IP `16.113.88.236`, and public IP `56.10.83.173`. The right pane shows the details of another EC2 instance with ID `i-0d06f5e2170bd9356`, name `ec2-user@ip-12-0-0-196`, private IP `56.10.83.173`, and public IP `56.10.83.173`. Both panes show the output of a `ping` command, indicating successful connectivity between the two instances. The terminal output for the first instance shows:

```
ec2-user@ip-10-0-1-223 ~$ ping 16.113.88.236
ping: 16.113.88.236: Name or service not known
ec2-user@ip-10-0-1-223 ~$ ping 16.113.88.236
PING 16.113.88.236 (16.113.88.236) 56(84) bytes of data:
64 bytes from 16.113.88.236: icmp_seq=61 ttl=117 time=44.2 ms
64 bytes from 16.113.88.236: icmp_seq=62 ttl=117 time=44.2 ms
64 bytes from 16.113.88.236: icmp_seq=63 ttl=117 time=44.2 ms
64 bytes from 16.113.88.236: icmp_seq=64 ttl=117 time=44.3 ms
64 bytes from 16.113.88.236: icmp_seq=65 ttl=117 time=44.5 ms
64 bytes from 16.113.88.236: icmp_seq=66 ttl=117 time=44.2 ms
64 bytes from 16.113.88.236: icmp_seq=67 ttl=117 time=44.2 ms
64 bytes from 16.113.88.236: icmp_seq=68 ttl=117 time=44.2 ms
64 bytes from 16.113.88.236: icmp_seq=69 ttl=117 time=44.2 ms
64 bytes from 16.113.88.236: icmp_seq=70 ttl=117 time=44.2 ms
64 bytes from 16.113.88.236: icmp_seq=71 ttl=117 time=44.2 ms
64 bytes from 16.113.88.236: icmp_seq=72 ttl=117 time=44.3 ms
64 bytes from 16.113.88.236: icmp_seq=73 ttl=117 time=44.2 ms
64 bytes from 16.113.88.236: icmp_seq=74 ttl=117 time=44.2 ms
64 bytes from 16.113.88.236: icmp_seq=75 ttl=117 time=44.2 ms
```

 The terminal output for the second instance shows:

```
ec2-user@ip-12-0-0-196 ~$ ping 56.10.83.173
PING 56.10.83.173 (56.10.83.173) 56(84) bytes of data:
64 bytes from 56.10.83.173: icmp_seq=1 ttl=116 time=44.2 ms
64 bytes from 56.10.83.173: icmp_seq=2 ttl=116 time=44.2 ms
64 bytes from 56.10.83.173: icmp_seq=3 ttl=116 time=44.2 ms
64 bytes from 56.10.83.173: icmp_seq=4 ttl=116 time=44.2 ms
64 bytes from 56.10.83.173: icmp_seq=5 ttl=116 time=44.2 ms
64 bytes from 56.10.83.173: icmp_seq=6 ttl=116 time=44.2 ms
64 bytes from 56.10.83.173: icmp_seq=7 ttl=116 time=44.2 ms
64 bytes from 56.10.83.173: icmp_seq=8 ttl=116 time=44.2 ms
64 bytes from 56.10.83.173: icmp_seq=9 ttl=116 time=44.2 ms
64 bytes from 56.10.83.173: icmp_seq=10 ttl=116 time=44.2 ms
64 bytes from 56.10.83.173: icmp_seq=11 ttl=116 time=44.2 ms
64 bytes from 56.10.83.173: icmp_seq=12 ttl=116 time=44.2 ms
64 bytes from 56.10.83.173: icmp_seq=13 ttl=116 time=44.2 ms
64 bytes from 56.10.83.173: icmp_seq=14 ttl=116 time=44.2 ms
64 bytes from 56.10.83.173: icmp_seq=15 ttl=116 time=44.2 ms
64 bytes from 56.10.83.173: icmp_seq=16 ttl=116 time=44.3 ms
64 bytes from 56.10.83.173: icmp_seq=17 ttl=116 time=44.2 ms
```

 The bottom of the console shows the instance details, including the VPC ID `vpc-055070ca2d7b85dc3`, Subnet ID `subnet-0b46daa5317abd75e`, and Instance ARN `arn:aws:ec2:ap-southeast-1:665511042079:instance/i-0d06f5e2170bd9356`. The AWS Compute Optimizer finding is also visible, showing an 'Opt-in to AWS Compute Optimizer for recommendation' link.

Figure 12: Successful private connectivity test

12. Result

The AWS VPC Peering configuration was successfully completed.

- Both VPCs were created successfully.
- Subnets were configured successfully.
- EC2 instances were deployed in the respective VPCs.
- VPC Peering Connection was successfully established.
- Peering connection status was verified as Active.
- Route tables were configured correctly.
- Security group rules were configured as required.
- Communication between the VPCs was successfully tested using private IP addresses.

13. Advantages of VPC Peering

VPC Peering provides private communication between VPCs without requiring traffic to pass through the public internet.

- Private communication between VPCs.
- Low-latency network connectivity.
- Improved network security.
- No requirement for public IP addresses for inter-VPC communication.
- Useful for connecting applications and services across separate VPCs.

14. Conclusion

This project successfully demonstrated the configuration of AWS VPC Peering between two independent VPCs. The required VPCs, subnets, EC2 instances, route tables, and security groups were configured to establish private network communication. The successful connectivity test verified that the two VPC environments could communicate through the VPC Peering Connection using private IP addresses. This project provides practical knowledge of AWS networking concepts, including VPCs, subnets, route tables, security groups, EC2 networking, and VPC Peering.

15. Project Evidence

The screenshots included in this document provide visual evidence of the AWS configurations and successful connectivity testing performed during the project.

Figure 13: Project evidence